(**Int**eger **Ex**tended Infinitely∞)

Arbitrary Precision Integer

Library

Mohammad KazemiMoghaddam

Version 2.0.1

intEx8

(**Int**eger **Ex**tended Infinitely∞)

Arbitrary Precision Integer

Library

Mohammad KazemiMoghaddam

Version 2.0.1

intEx8: Integer Extended Infinitely∞

Overview

intEx8 is a **C library** providing functionality for **arbitrary-precision integer arithmetic**. It supports:

- Arithmetic operations (with both **intx**, **int64**): `+`, `-`, `*`, `/`, `%`, `abs`, `negate`
- Augmented assignment operations (with both **intx**, **int64**): `+=`, `-=`, `*=`, `/=`, `%=`
- Comparison Operations (with both **intx**, **int64**): `==`, `<`, `<=`, `>`, `>=`

Terminology

- **intx**: Represents a big integer (`typedef {dig_t *ptr; int64_t size; } intx_t`).
- **int64:** Represents a primitive integer (`int64_t`).
- **digit**: A 32-bit integer unit (`dig_t`) used to store digits of **intx**s internally.
- **Base Representation**: Integers are stored as **arrays of 32-bit digits** in base `2^32`.
- **Sign Representation**: Sign of `size` member of intx_t structure determines the sign.

Interfaces in intEx8

intEx8 provides **two interfaces**:

1. ix8_

- Allocates memory for results automatically.
- Developers must **free memory** using `ix8_free()` or `ix8_free_s()`.
- Implemented in `ix8.c` and `ix8.h`.

Function list:

Name	Operation	C Syntax	Note
ix8_copy	y=x	y = ix8_copy(x);	Deep copy
ix8_copy_i	x=i	x = ix8_copy_i(i);	From `int64_t`
ix8_copy_s	x=atoi(s)	x = ix8_copy_s("123");	From decimal string
ix8_copy_to_s	s=itoa(x)	s = ix8_copy_to_s(x);	To string; use `ix8_free_s()`
ix8_add	z=x+y	z = ix8_add(x, y);	Addition
ix8_add_i	z=x+i	z = ix8_add_i(x, i);	Add with `int64_t`
ix8_addeq	x+=y	ix8_addeq(&x, y);	In-place addition

ix8_addeq_i	x+=i	ix8_addeq_i(&x, i);	In-place add `int64_t`
ix8_sub	z=x-y	z = ix8_sub(x, y);	Subtraction
ix8_sub_i	z=x-i	z = ix8_sub_i(x, i);	Subtract `int64_t`
ix8_i_sub	z=i-x	z = ix8_i_sub(i, x);	`int64_t` minus big int
ix8_subeq	x-=y	ix8_subeq(&x, y);	In-place subtraction
ix8_subeq_i	x-=i	ix8_subeq_i(&x, i);	In-place subtract `int64_t`
ix8_mul	z=x*y	z = ix8_mul(x, y);	Multiplication
ix8_mul_i	z=x*i	z = ix8_mul_i(x, i);	Multiply by `int64_t`
ix8_mul_p2	z=x*sgn(y)*2^|y|	z = ix8_mul_p2(x, y);	Multiply by power of 2
ix8_muleq	x*=y	ix8_muleq(&x, y);	In-place multiplication
ix8_muleq_i	x*=i	ix8_muleq_i(&x, i);	In-place multiply `int64_t`
ix8_muleq_p2	x*=sgn(y)*2^|y|	ix8_muleq_p2(&x, y);	In-place multiply by power of 2
ix8_div	z=x/y	z = ix8_div(x, y);	Division
ix8_div_i	z=x/i	z = ix8_div_i(x, i);	Divide by `int64_t`
ix8_i_div	z=i/x	z = ix8_i_div(i, x);	`int64_t` divided by big int
ix8_div_p2	z=x/(sgn(y)*2^|y|)	z = ix8_div_p2(x, y);	Divide by power of 2
ix8_diveq	x/=y	ix8_diveq(&x, y);	In-place division
ix8_diveq_i	x/=i	ix8_diveq_i(&x, i);	In-place divide `int64_t`
ix8_i_diveq	i/=x	ix8_i_diveq(&x, i);	`int64_t` divide by big int
ix8_diveq_p2	x/=sgn(y)*2^|y|	ix8_diveq_p2(&x, y);	In-place divide by power of 2
ix8_mod	z=x%y	z = ix8_mod(x, y);	Modulo
ix8_mod_i	z=x%i	z = ix8_mod_i(x, i);	Modulo `int64_t`
ix8_i_mod	z=i%x	z = ix8_i_mod(i, x);	`int64_t` mod big int
ix8_mod_p2	z=x%(2^y)	z = ix8_mod_p2(x, y);	Modulo power of 2
ix8_modeq	x%=y	ix8_modeq(&x, y);	In-place modulo
ix8_modeq_i	x%=i	ix8_modeq_i(&x, i);	In-place modulo `int64_t`
ix8_i_modeq	i%=x	ix8_i_modeq(&x, i);	In-place `int64_t` mod big int
ix8_modeq_p2	x%=2^y	ix8_modeq_p2(&x, y);	In-place modulo power of 2
ix8_negate	y=-x	y = ix8_negate(x);	Negation
ix8_negate_me	x=-x	ix8_negate_me(&x);	In-place negation
ix8_abs	y=|x|	y = ix8_abs(x);	Absolute value
ix8_abs_me	x=|x|	ix8_abs_me(&x);	In-place absolute value
ix8_eq	x==y	if(ix8_eq(x, y);	Equality
ix8_le	x<=y	if(ix8_le(x, y)) {}	Less than or equal to
ix8_ge	x>=y	if(ix8_ge(x, y)) {}	Greater than or equal to
ix8_lt	x<y	if(ix8_lt(x, y)) {}	Less than
ix8_gt	x>y	if(ix8_gt(x, y)) {}	Greater than
ix8_is_zero	x==0	if(ix8_is_zero(x)) {}	Zero check
ix8_gt_zero	x>0	if(ix8_gt_zero(x)) {}	Positive check
ix8_lt_zero	x<0	if(ix8_lt_zero(x)) {}	Negative check
ix8_eq_i	x==i	if(ix8_eq_i(x, i)) {}	Equality to `int64_t`
ix8_le_i	x<=i	if(ix8_le(x, i)) {}	Less than or equal to `int64_t`
ix8_ge_i	x>=i	if(ix8_ge(x, i)) {}	Greater than or equal to `int64_t`
ix8_lt_i	x<i	if(ix8_lt(x, i)) {}	Less than `int64_t`
ix8_gt_i	x>i	if(ix8_gt(x, i)) {}	Greater than `int64_t`

2. i8_

- Requires the caller to **allocate and manage memory**.
- Offers better performance but requires manual memory management.
- Implemented in `i8.c` and `i8.h`.

Function list:

Name	Operation	C Syntax	Note
i8_copy	y=x	y = i8_copy(x, ..);	Copy big int to caller's buffer
i8_copy_i	x=i	x = i8_copy_i(i, ..);	From `int64_t`
i8_copy_s	x=atoi(s)	x = i8_copy_s("123", ..);	From decimal string
i8_copy_to_s	s=itoa(x)	s = i8_copy_to_s(x, ..);	Convert to string
i8_add	z=x+y	z = i8_add(x, y, ..);	Addition
i8_add_i	z=x+i	z = i8_add_i(x, i, ..);	Add with `int64_t`
i8_sub	z=x-y	z = i8_sub(x, y, ..);	Subtraction
i8_sub_i	z=x-i	z = i8_sub_i(x, i, ..);	Subtract `int64_t`
i8_i_sub	z=i-x	z = i8_i_sub(i, x, ..);	`int64_t` minus big int
i8_mul	z=x*y	z = i8_mul(x, y, ..);	Multiplication
i8_mul_i	z=x*i	z = i8_mul_i(x, i, ..);	Multiply with `int64_t`
i8_mul_p2	z=x*sgn(y)*2^|y|	z = i8_mul_p2(x, y, ..);	Multiply by power of 2
i8_div	z=x/y	z = i8_div(x, y, ..);	Division
i8_div_p2	z=x/(sgn(y)*2^|y|)	z = i8_div_p2(x, y, ..);	Divide by power of 2
i8_mod	z=x%y	z = i8_mod(x, y, ..);	Modulo
i8_mod_p2	z=x%(2^y)	z = i8_mod_p2(x, y, ..);	Modulo power of 2
i8_negate	y=-x	y = i8_negate(x, ..);	Negation
i8_negate_me	x=-x	i8_negate_me(&x);	In-place negation
i8_abs	y=|x|	y = i8_abs(x, ..);	Absolute value
i8_abs_me	x=|x|	i8_abs_me(&x);	In-place absolute
i8_eq	x==y	if(i8_eq(x, y)) {}	Equality
i8_le	x<=y	if(i8_le(x, y)) {}	Less than or equal to
i8_ge	x>=y	if(i8_ge(x, y)) {}	Greater than or equal to
i8_lt	x<y	if(i8_lt(x, y)) {}	Less than
i8_gt	x>y	if(i8_gt(x, y)) {}	Greater than
i8_is_zero	x==0	if(i8_is_zero(x)) {}	Zero check
i8_gt_zero	x>0	if(i8_gt_zero(x)) {}	Positive check
i8_lt_zero	x<0	if(i8_lt_zero(x)) {}	Negative check
i8_eq_i	x==i	if(i8_eq_i(x, i)) {}	Equality to `int64_t`
i8_le_i	x<=i	if(i8_le(x, i)) {}	Less than or equal to `int64_t`
i8_ge_i	x>=i	if(i8_ge(x, i)) {}	Greater than or equal to `int64_t`
i8_lt_i	x<i	if(i8_lt(x, i)) {}	Less than `int64_t`
i8_gt_i	x>i	if(i8_gt(x, i)) {}	Greater than `int64_t`

String Conversion Utilities

Convert Big Integer to Decimal String

```
char *ix8_copy_to_s(intx_t x);
```

- Converts a big integer to a **decimal string representation**.
- The returned string **must be freed** using `ix8_free_s()`.

Initialize a Big Integer from a Decimal String

```
intx_t ix8_copy_s(const char *str);
```

Example

```
intx_t x = ix8_copy_s("290293826899404773098674720100 83");
...
ix8_free(x);
```

- More initialization methods will be added in future versions.

Error Handling

- Both **`ix8_`** and **`i8_`** interfaces use `int intEx8_errno` to store error codes.
- Error codes are defined in `intEx8.h`.

Using intEx8 in a Project

1. Add the following files to your project/build system:
    - `ix8.c, i8.c, toom3_multiply.c, util.c`
2. Include the required header file:
    - `#include "intEx8.h"` to call library functions.
3. Call `intEx8_init()` to initialize the library (Or equivalently, write `intEx8_errno = INTEX8_OK`).

Multiplication and Division

Multiplication Limits

- The **maximum supported operand size** is limited (defined in `intEx8.h`) by `INTEX8_MAX_MULTIPLICATION_DIGITS`.
- This value **can be increased**, but increasing it also **increases the buffer size** for the Toom-3 algorithm implemented in `_toom3_multiply()`.

Division and Remainder Limits

- The **dividend size** is limited by `INTEX8_MAX_DIVIDEND_DIGITS` (defined in `intEx8.h`). Developers **can increase** it if larger size for dividend is needed.

Best Practices for Using intEx8

- ✓ **Always check `intEx8_errno` after calling functions**. If it is not `INTEX8_OK`, the return value is invalid. *Note* that intEx8 library functions DO NOT set `intEx8_errno` to `INTEX8_OK` as initial value.
- ✓ **Avoid recursive function calls** such as:

```
ix8_add(ix8_multiply(a, b), y); // ⚠ Memory leak!
```

- ✓ **Avoid assigning results to the same variable**:

```
x = ix8_add(x, y); // ⚠ Memory leak! Use ix8_addeq(&x, y) instead
y = ix8_add(x, y); // ⚠ Memory leak! Use ix8_addeq(&y, x) instead
```

Instead, store the result in a temporary variable and **free memory before reassignment**.

Advanced Topics

Memory Allocation Considerations

- The `ptr` member of `intx_t` may be allocated with **more digits than** `size` member.
- Use utility functions to determine **actual allocated memory**.

Handling Sign Extension

- intEx8 does not provide `unsigned intx_t`.

Example

```
x = {{18, 750198, 51247408}, -3}; // x is a negative intx
y = {{18, 750198, 51247408}, 3}; // y is a positive intx equal to -x
```

Future Improvements

Bug Fixes – Any reported issues will be addressed.

C++ Interface – A C++ wrapper class for `i8_` functions with overloaded `+`, `-`, `*`, `/`, `%` operators.

Floating-Point Support – Arbitrary-precision floating-point arithmetic.

Fractional Number Support – Exact representation of rational numbers.

Performance Enhancements – Further optimizations for large-number calculations.

Conclusion

intEx8 is a **high-performance** arbitrary-precision integer library designed for **efficiency, flexibility, and scalability**. With its **two interface sets**, robust error handling, and powerful mathematical capabilities, it is an ideal choice for developers working with **large integers beyond fixed-size types**.